Movie Recommender

Hasti Javadzadeh-401222039

Dataset

The initial dataset contained metadata(45000 movies), keywords(JSON), credits, links(tmdb and imdb of all movies), small_links(tmdb and imdb of 9000 movies), ratings, ratings_small(100000 ratings from 700 users on 9000 movies).

In the preprocessing step, I first did the data cleaning (removing duplicate and null values). Then I chose the most informative features of movies.

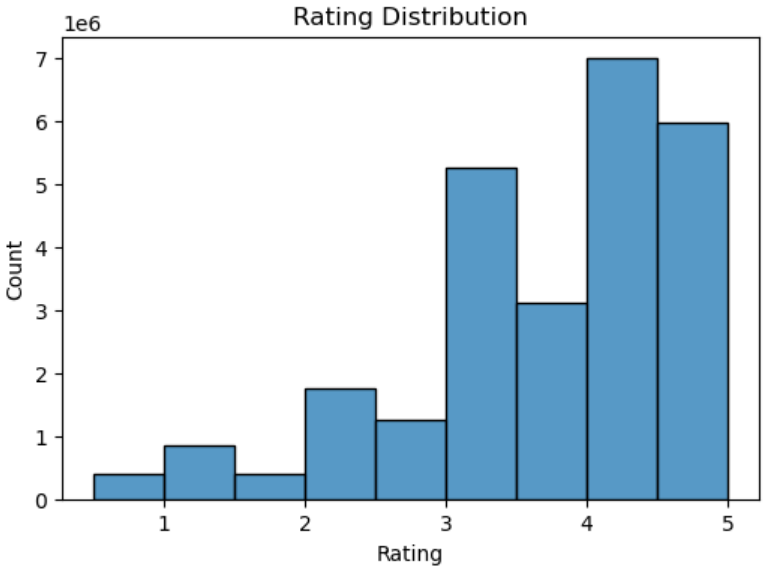
I also cleaned credits and keywords datasets and merged movies with them. From the cast I only kept the first three and from the crew I kept the director.

I built a meta tags column for each movie. It contains genres, overview, tagline, release_decade(which I generate using release year), keywords, cast, director.

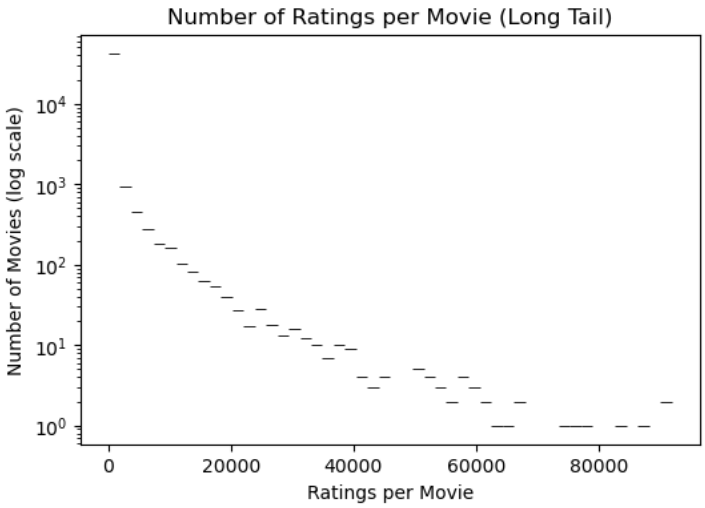
I parsed genres, company, country, cast, keywords.

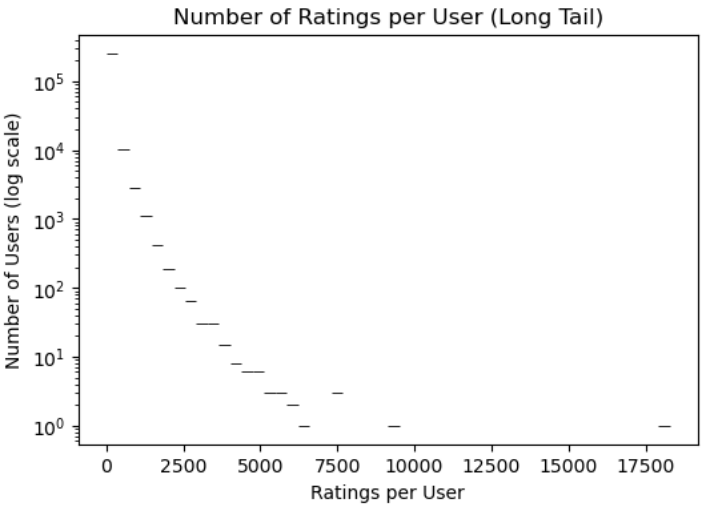
EDA

I explored the rating distribution, long tails, etc.

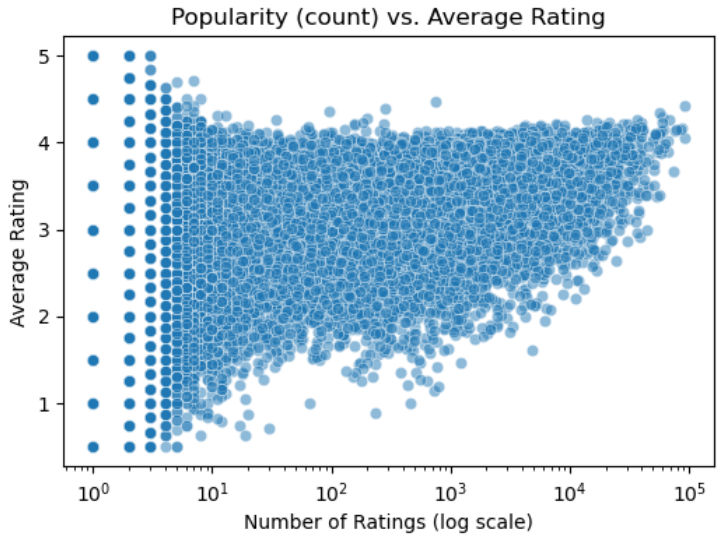


On the left (of the figure number of ratings per movie) you see most movies that have only a handful of ratings. On the right, a tiny fraction of movies have tens of thousands of ratings. The distribution of ratings per movie is highly skewed; most movies receive very few ratings, while a small set of popular titles dominate the dataset.

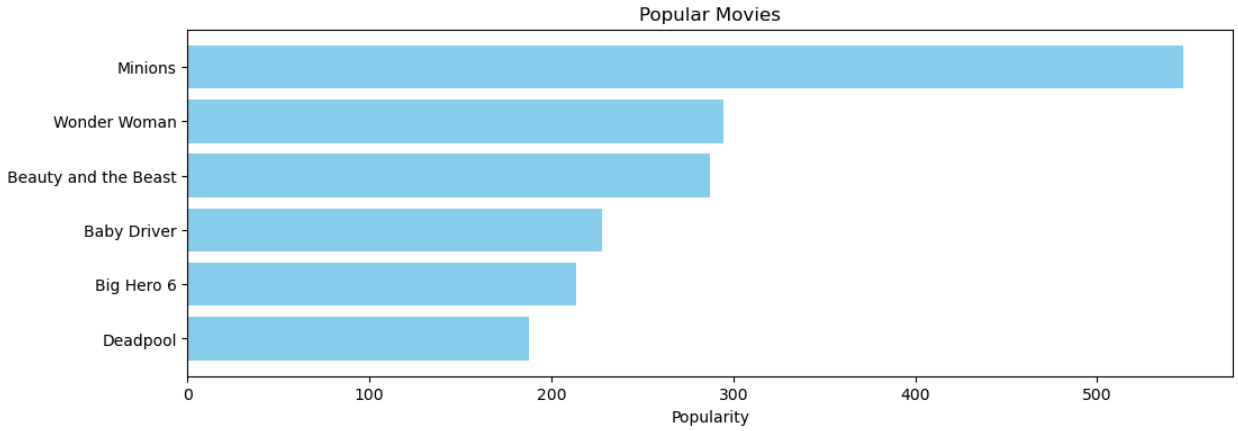




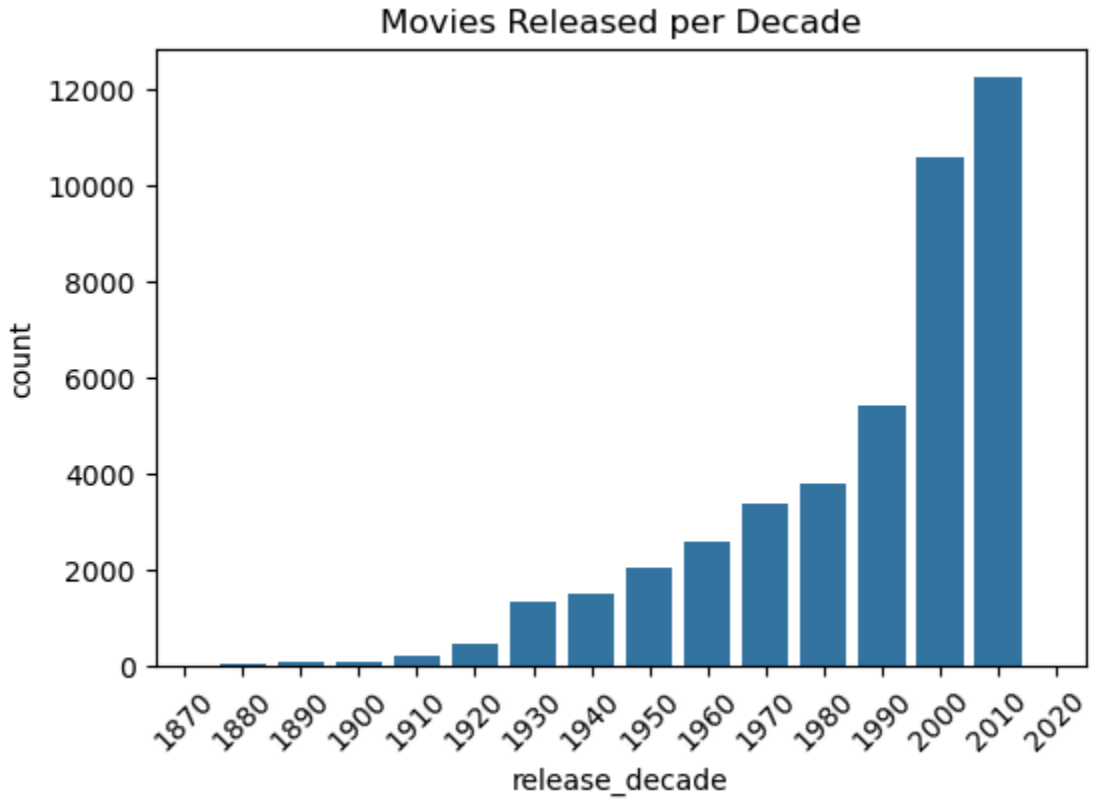
The relationship between popularity and average rating is highly skewed. Movies with very few ratings display extreme average scores, while movies with thousands of ratings stabilize around average values. This highlights the need for weighted popularity baselines such as IMDb's WR formula.



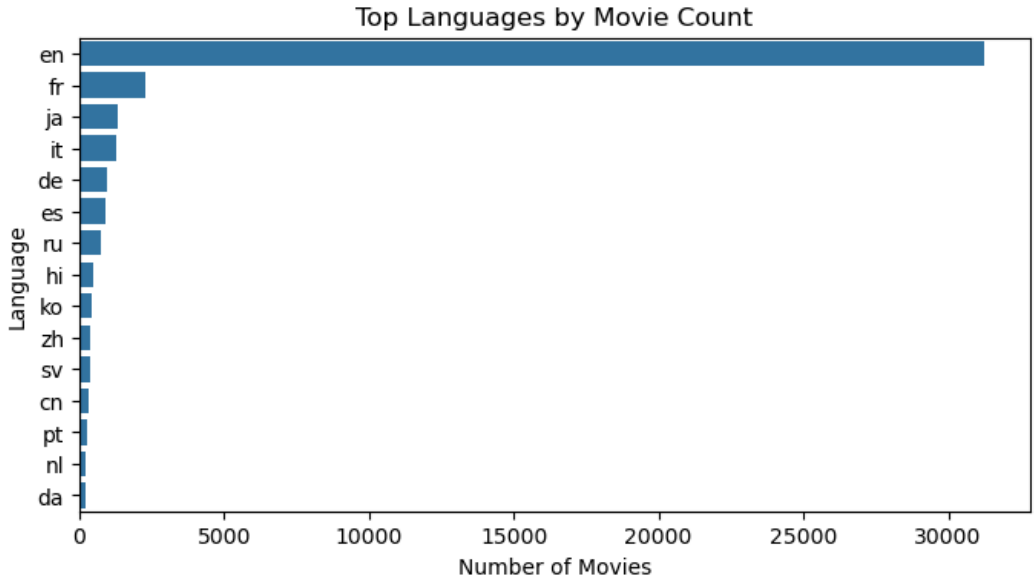
The figure below shows the popular movies according to the popularity metric of TMDb.

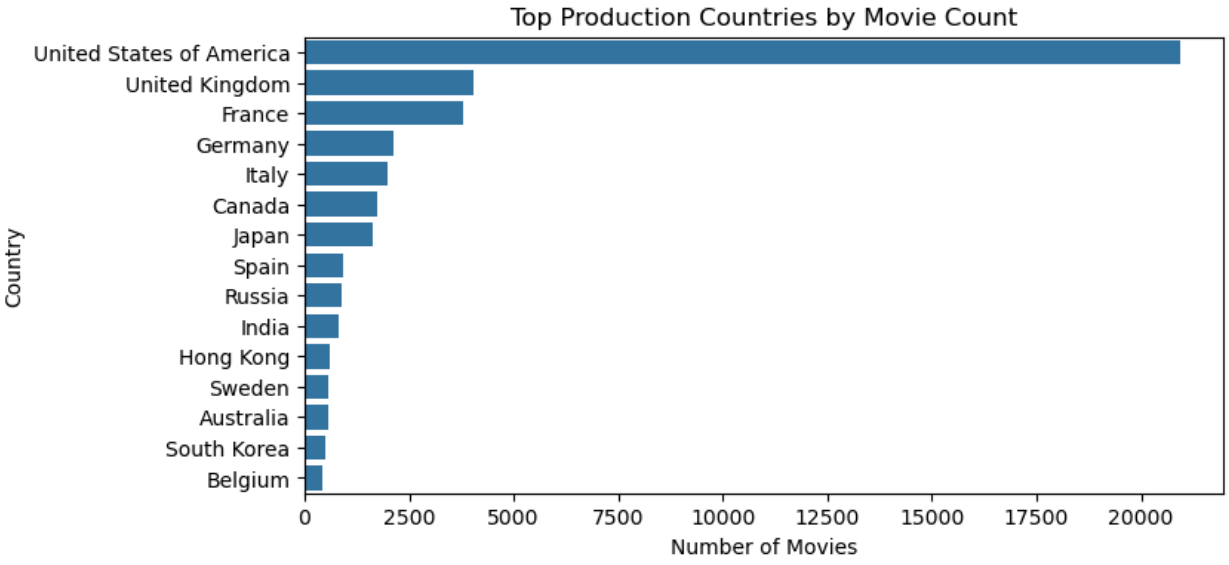


The figure below indicates that the number of movies released each decade has increased.



The top language is English and USA has produced the most moveis:





The user-item interaction matrix is extremely sparse. This is a common challenge, as most users interact with only a tiny fraction of the catalog.

Number of users: 270896
Number of movies: 45115
Number of ratings: 26024289
Sparsity: 0.9979 (99.79%)

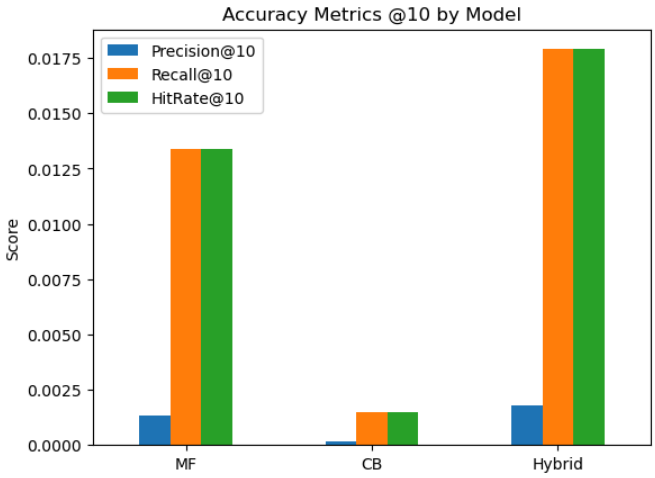
Baselines
First I computed movie stats for movies (their average rating, number of ratings). The first recommender recommends top movies based on the average rating which is unfair since number of votes is not considered. The next baseline I implemented used IMDb weighted rating and corrected the issue of baseline 1.

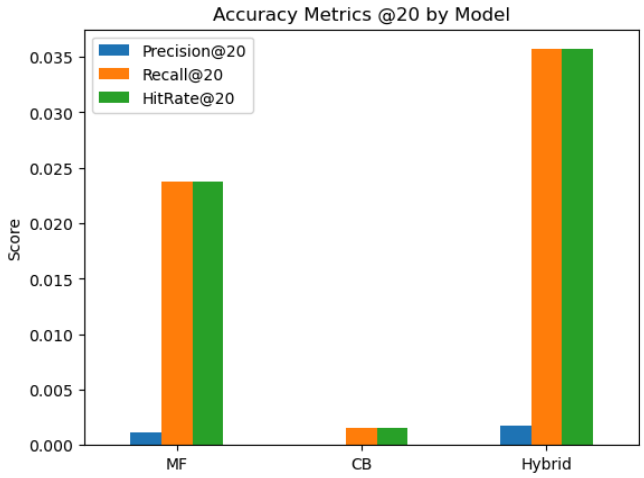
Content Based
I built a tf_matrix and a cv_matrix with 5000 features. Then I reduced dimensionality with truncated SVD. recommendation using tf_matrix is very precise, but using reduced_matrix is a bit more exploratory.
I also built user profiles (gives a vector which is the mean of the movies they watched) and did the recommendation using the seen items. The recommender also shows the shared genres (and cast if any).

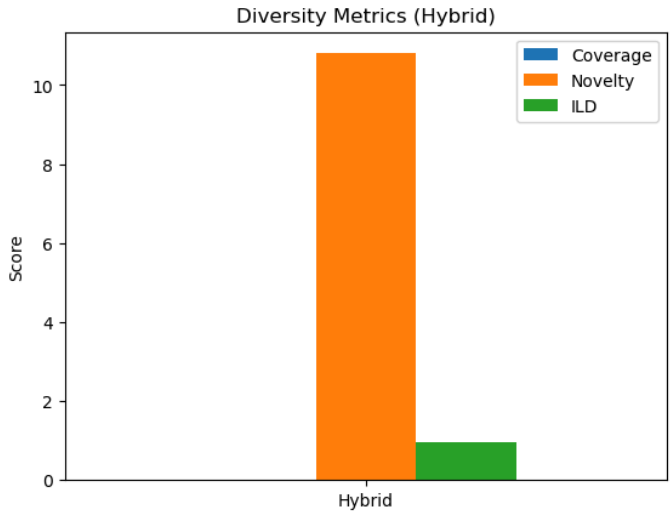
Collaborative Filtering
I implemented item-item k-NN. I built a user-item matrix with users being rows, movies being columns, and ratings being the values. Then computed the cosine similarity and stored the top 10 similar movies for each movie. For a given use, it recommends movies similar to those they liked (rated high).
Next I implemented Marix Factorisation using Suorise's svd.

Hybrid Model
Blends strengths of MF(SVD) and content-based. Each model's scores were normalized to **[0,1]** per user. The MF model gives a predicted rating per candidate. Content gives cosine similarity to user profile vectors. Then final = α * mf + β * content was calculated.

Evaluation
For each user I held out their most recent rating (with respect to timestamps) as the test item (leave-one-out). All other ratings became train.
I implemented three metrics: Precision@K, Recall@K, Hit Rate@K.







LOO evaluation is very strict. Each user has only 1 test item.. Precision@K = fraction of recommended items that match exactly that 1 item. With thousands of movies, even a very good recommender will rarely put the exact LOO movie in the top-10. Thus, numbers like 0.001–0.002 Precision@10 are normal for LOO on large datasets.

I calculated catalog coverage (0.012), novelty (10.78), and intra-list diversity (0.96) for the hybrid model.
Coverage (1.2%) → only a small slice of catalog is recommended.
Novelty (10.8) → within that slice, the items are relatively "rare" (not just blockbusters).
ILD (0.97) → the lists are very diverse (no "clones" in top-K).
So the hybrid system is precise but narrow, while still producing novel and diverse recommendations — which is actually a typical tradeoff in recommender systems.

I also provided 95% confidence intervals:

Hybrid model:

Precision@20: 0.0018 (95% CI 0.0011, 0.0025)

Recall@20: 0.0358 (95% CI 0.0224, 0.0507)

HitRate@20: 0.0358 (95% CI 0.0224, 0.0507)

MF model:

Precision@20: 0.0012 (95% CI 0.0006, 0.0018)

Recall@20: 0.0238 (95% CI 0.0119, 0.0358)

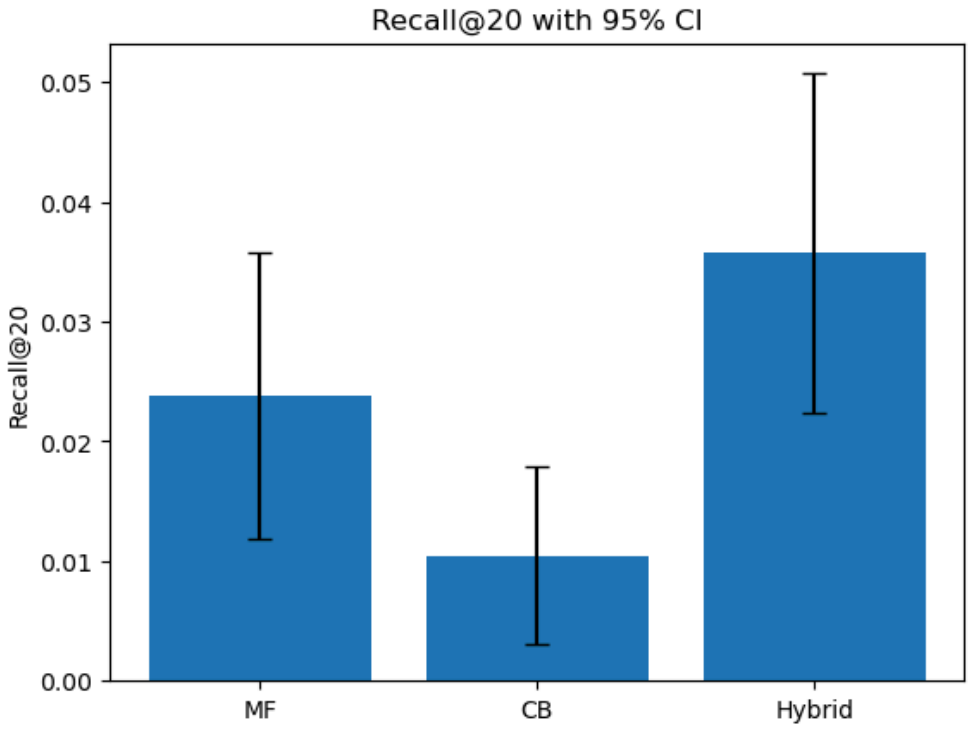
HitRate@20: 0.0238 (95% CI 0.0119, 0.0358)

Content based model:

Precision@20: 0.0005 (95% CI 0.0001, 0.0009)

Recall@20: 0.0104 (95% CI 0.0030, 0.0179)

HitRate@20: 0.0104 (95% CI 0.0030, 0.0179)



The link to the live Hugging Face Space:

https://huggingface.co/spaces/hastijavadzadeh/movie-recommender